

Lesson 5: Navigation & Routes

Why screens matter

Real apps have more than one screen. A shop app has a product list, a product page, and a cart. A learning app has a home, lessons, and quizzes -- like the one you are using right now.

In Flutter, moving between screens is called navigation. It is handled by a widget called the Navigator. Think of it as a stack of cards: you push a new screen on top, and pop it off to go back.

Pushing a new screen

To open a new screen, you push a route onto the navigator:

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => const DetailsScreen()),  
);
```

Three things to notice:

- context -- every widget's build() gets one. The navigator is found through the context, so you always push from inside a widget.
- MaterialPageRoute -- this controls *how* the new screen shows up (it slides in, like most Android apps).
- builder -- a small function that returns the widget for the new screen.

That is it. One call, and your new screen slides in.

Going back

To return to the last screen, pop the current one off the stack:

```
Navigator.pop(context);
```

This is often linked to a back button. Flutter even adds a back button to your AppBar on its own when there is a screen to go back to.

Passing data forward

Screens often need to share data. A product list needs to tell the details screen *which* product to show. You do this through the constructor:

```
// On the first screen:
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => DetailsScreen(productId: 42),
  ),
);

// The details screen takes it in:
class DetailsScreen extends StatelessWidget {
  final int productId;

  const DetailsScreen({super.key, required this.productId});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Details')),
      body: Center(child: Text('Product #${productId}')),
    );
  }
}
```

This is the same idea from Lesson 2: data goes down through constructors. Navigation just adds the screen change around it.

Getting a result back

At times the new screen needs to send something back -- like a settings page sending back the user's choice. `pop()` can take an optional result:

```
// On the second screen (sending back):
Navigator.pop(context, 'Dark mode');

// On the first screen (getting it):
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const SettingsScreen()),
);
print(result); // 'Dark mode'
```

See the `await`: `push()` gives back a `Future` that finishes when the screen is popped. You will learn more about futures in Lesson 10 -- for now, just remember this pattern.

Named routes (a preview)

When your app has many screens, writing `MaterialPageRoute` each time gets boring. Flutter lets you

name your routes instead:

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const HomeScreen(),  
    '/details': (context) => const DetailsScreen(),  
  },  
);
```

Then move between screens with just the name:

```
Navigator.pushNamed(context, '/details');
```

You do not need this yet -- but you will like it when your app has ten screens.

Key terms

- Navigator -- the widget that manages your stack of screens.
- Route -- one screen (page) in the navigation stack.
- Push -- put a new screen on top of the stack.
- Pop -- take the top screen off and go back.
- MaterialPageRoute -- a route with the normal Material page slide-in.
- BuildContext (context) -- how widgets find the navigator above them.